ARCHITECTURE HLD DOCUMENT — Sample Social Media Platform
==========

System Overview:
A scalable social media platform targeting 1 million daily active users (DAU).
The system supports user authentication, social feed, real-time chat,
media uploads, and push notifications.

Current Tech Stack:
- Frontend: React.js (SPA)
- Backend: Node.js / Express (Monolith)
- Database: MySQL (single instance, no replication)
- File Storage: Local disk (no CDN)
- Hosting: Single EC2 t2.medium instance on AWS
- No caching layer, no message queue, no load balancer

Current Scale:
- 50,000 registered users
- 5,000 daily active users
- Average response time: 800ms
- Frequent timeout errors during peak hours

Known Bottlenecks and Issues:
1. Single point of failure — one server handles all traffic
2. No horizontal scaling capability
3. Database bottleneck — N+1 queries on feed page
4. No CDN — media served directly from EC2, high latency globally
5. No caching — every request hits the database
6. No rate limiting — susceptible to DDoS
7. No message queue — notification sending is synchronous and blocks requests
8. No monitoring or alerting system
9. Authentication is session-based using in-memory store (lost on restart)
10. No CI/CD pipeline — manual deployments with downtime

Target Requirements:
- Scale to 1,000,000 DAU
- 99.9% uptime SLA
- Sub-200ms p95 API latency
- Global CDN for media delivery
- Real-time chat with WebSocket support
- Push notifications at scale
- GDPR compliance for EU users

Proposed Services:
- Auth Service: JWT-based stateless authentication, Redis session store

- User Service: Profile management, follows/followers graph
- Feed Service: Fan-out on write for users with <10K followers, fan-out on read for celebrities
- Chat Service: WebSocket server with Redis pub/sub for horizontal scaling
- Media Service: S3 upload, CloudFront CDN, video transcoding via Lambda
- Notification Service: Firebase Cloud Messaging, async via SQS queue

Proposed Infrastructure:
- AWS Application Load Balancer with auto-scaling groups
- CloudFront CDN for static assets and media
- Route 53 for DNS
- RDS PostgreSQL with read replicas for user data
- DynamoDB for chat messages (high write throughput)
- ElastiCache Redis for session store, rate limiting, and feed caching
- Amazon SQS for async notification jobs
- ELK Stack for logging and monitoring
- Prometheus + Grafana for metrics

Data Layer:
- Write DB: RDS PostgreSQL (primary) for user profiles, posts, relationships
- Read DB: RDS PostgreSQL read replicas for feed reads
- Cache: Redis ElastiCache cluster for hot posts and user sessions
- NoSQL: DynamoDB for chat messages (unlimited scale)
- Object Storage: S3 with lifecycle policies, CloudFront distribution
- Search: Elasticsearch for full-text user/content search

Security:
- WAF (Web Application Firewall) on CloudFront
- JWT with short expiry (15 min) + refresh token rotation
- Rate limiting via Redis token bucket algorithm
- KMS encryption for sensitive fields (phone, email)
- VPC with private subnets for all backend services

Scalability Notes:
- Stateless services allow horizontal scaling via ASG
- Database sharding by user_id for large scale
- CDN reduces origin server load by 70-80%
- SQS decouples notification service from API latency
- Redis handles 100K+ reads/sec for feed caching